# SIMATS ENGINEERING
## ASSESSMENT TOOL 2

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

***CO1:*** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*( BL3)*

*Assessment Tool Weightage: 35%*

---

**QUESTIONS: 3** **Duration : 1 Hour**
**Total Marks:30**

Annexure A

## Constraint-Based Problem Solving

---

***Code of Conduct:***

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

Annexure A

# Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Problem Understanding | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understanding; misses key constraints | Fails to identify constraints correctly |
| Constraint Analysis | 25% | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization | No meaningful analysis or prioritization |
| Solution Development | 25% | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts |
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided |

**SIMATS ENGINEERING**
**ASSESSMENT TOOL 2 – ANSWER SHEET**
Artificial Intelligence | CSA17
Constraint-Based Problem Solving

**Name: Aravindh. R** **Register No.:** 192312024

## 1. Hospital Doctor–Shift Assignment Using Backtracking

Problem understanding: Assign D1, D2 and D3 to Morning, Afternoon and Night, with exactly one doctor per shift and one shift per doctor. D1 cannot work Night; D2 must work earlier than D3; D3 cannot work Morning.

Method: Backtracking assigns a shift, checks all applicable constraints, and reverses a choice if it leads to a conflict. Interpret D2 before D3 according to Morning < Afternoon < Night.

| Step | Trial | Constraint check | Decision |
|---|---|---|---|
| 1 | D1 → Morning | D1 is not on Night. | Accept |
| 2 | D2 → Afternoon | D2 can still be earlier than D3. | Accept provisionally |
| 3 | D3 → Night | D3 is not Morning; D2 is before D3; shifts are distinct. | Accept |
| 4 | Complete schedule | Each doctor has one shift; each shift has one doctor; all constraints hold. | Goal reached |

**Final valid schedule:** Morning – D1; Afternoon – D2; Night – D3.

Interpretation: This branch satisfies every constraint, so no backtracking is needed. If a trial violated a constraint, undo it and try another assignment.

## 2. 5×5 Grid: BFS and Manhattan Distance

Important limitation: The uploaded assessment states that a 5×5 grid contains S, G and obstacles, but the pages do not show the actual grid or obstacle coordinates. Therefore, the exact expansion order, optimal path and cost cannot be determined uniquely. The grid below is an illustrative example only.

| | C1 | C2 | C3 | C4 | C5 |
|---|---|---|---|---|---|
| R1 | S | · | · | # | · |
| R2 | # | # | · | # | · |
| R3 | · | · | · | · | · |
| R4 | · | # | # | # | · |
| R5 | · | · | · | G | · |

Legend: S=start, G=goal, #=obstacle, ·=free cell. Coordinates use (row, column), beginning at (1,1).

Method: BFS uses a FIFO queue and finds a minimum-move path when every move costs 1. Manhattan distance is $h(n)=|row(n)-row(G)|+|column(n)-column(G)|$. It can be recorded as a heuristic, but standard BFS does not use it to prioritize nodes; Greedy Best-First Search prioritizes h(n), while A* prioritizes g(n)+h(n).

### BFS steps

1. Initialize the queue with S, mark it visited, and set parent(S)=NONE.
2. Remove the front node and examine neighbours in the fixed order Up, Down, Left, Right.
3. Reject out-of-grid cells, obstacles and visited cells. Mark each accepted neighbour visited, record its parent, and enqueue it.
4. Continue until G is reached. Reconstruct the path by following parent links backwards.
5. Calculate total cost as the number of moves because each move costs 1.

| Expansion | New neighbours (U,D,L,R order) | Queue after expansion (front → rear) |
|---|---|---|

| | | |
|---|---|---|
| S (1,1), h=6 | (1,2) | (1,2) |
| (1,2), h=5 | (1,3) | (1,3) |
| (1,3), h=4 | (2,3) | (2,3) |
| (2,3), h=3 | (3,3) | (3,3) |
| (3,3), h=2 | (3,2), (3,4) | (3,2), (3,4) |
| (3,2), h=3 | (3,1) | (3,4), (3,1) |
| (3,4), h=1 | (3,5) | (3,1), (3,5) |
| (3,1), h=4 | (4,1) | (3,5), (4,1) |
| (3,5), h=2 | (2,5), (4,5) | (4,1), (2,5), (4,5) |
| (4,1), h=5 | (5,1) | (2,5), (4,5), (5,1) |
| (2,5), h=3 | (1,5) | (4,5), (5,1), (1,5) |
| (4,5), h=1 | (5,5) | (5,1), (1,5), (5,5) |
| (5,1), h=4 | (5,2) | (1,5), (5,5), (5,2) |
| (1,5), h=4 | None | (5,5), (5,2) |
| (5,5), h=2 | (5,4)=G | (5,2), G |
| (5,2), h=3 | (5,3) | G, (5,3) |

Illustrative optimal route: S (1,1) → (1,2) → (1,3) → (2,3) → (3,3) → (3,4) → (3,5) → (4,5) → (5,5) → G (5,4). Total cost = 9 moves.

Interpretation: BFS is optimal for equal step costs. Since the actual grid is absent from the assessment pages, this path and cost are illustrative and should not be treated as the unique answer to the original question.

## 3. Autonomous Rescue Robot

### (a) Constraint analysis

- **Movement:** Only Up, Down, Left and Right are permitted; diagonal movement is not allowed.
- **Obstacles:** Blocked cells must never be included in a route.
- **Partial observability:** The robot sees adjacent cells only, so it must maintain an internal map of known free, blocked, risky and unknown cells.
- **Costs:** An ordinary move costs 1. Entering a risky cell costs 1+2=3.
- **Safety and optimality:** The robot must avoid blocked/unsafe routes and minimize the combined movement and risk cost; the shortest-distance route may not be the cheapest.
- **Online updates:** Newly sensed information can change the map, so the robot must update its knowledge and re-plan.

### (b) Approach: Online Search Agent + Uniform Cost Search

Use an online agent to sense adjacent cells and update its internal map, combined with UCS to find the lowest-cost route through the currently known traversable cells. UCS expands the frontier node with the lowest accumulated cost g(n).

6. Start at S and initialize the internal map with currently known information.
7. Sense the four adjacent cells and label them free, blocked or risky; keep unseen cells unknown.
8. Run UCS on the known map using step cost = 1 + risk penalty, where the penalty is 2 for entering a risky cell and 0 otherwise.
9. Expand the lowest-cost frontier node. Ignore blocked cells and update a node's parent/cost whenever a cheaper route is found.
10. Move one step along the selected route, sense again and update the map.
11. If new information invalidates or changes the route, run UCS again from the current position.
12. If no known route exists, explore a safe reachable frontier. Stop when G is reached or report that no safe known route is available.

**Pseudocode:**

```
WHILE current_position != G:
```

```
    observations = SENSE_ADJACENT_CELLS()
    UPDATE_INTERNAL_MAP(observations)
    path = UCS(known_map, current_position, G,
               cost = 1 + risk_penalty(destination_cell))
    IF path exists:
        MOVE_ONE_STEP(path)
    ELSE:
        frontier = FIND_SAFE_REACHABLE_UNKNOWN_FRONTIER()
        IF frontier does not exist:
            REPORT "No safe known route"; STOP
        MOVE_ONE_STEP(LOWEST_COST_ROUTE_TO(frontier))
    current_position = GET_POSITION()
REPORT "Survivor reached"
```

UCS is optimal over the current known map because all costs are non-negative. In a partially observed environment, global optimality cannot be guaranteed until enough of the building is explored.

### (c) AI concepts

- **Intelligent agent:** The robot senses surroundings, updates internal knowledge, selects actions and moves toward the survivor.
- **Rationality:** A rational action maximizes expected success and safety using available percepts and knowledge. The robot avoids known obstacles and accounts for risk cost.
- **Environment type:** Partially observable, sequential, discrete (grid cells and four actions), and initially unknown/dynamic as discoveries update the map. Costs are deterministic once risk status is known, but observations may be uncertain.

### (d) Justification

Online sensing is suitable because the robot cannot observe the whole building at once. UCS is suitable because routes have different non-negative costs due to risky zones, unlike ordinary BFS which treats all moves equally. Combining them lets the robot update its map, select the least-cost known route and re-plan when it discovers new obstacles. Explicit safety rules are still necessary; cost minimization alone does not guarantee safety.

## Conclusion

Backtracking finds a feasible doctor schedule while enforcing all constraints. BFS finds a minimum-move path when every move has equal cost; Manhattan distance is a heuristic but does not guide standard BFS expansion. For rescue navigation, online sensing combined with UCS supports adaptive least-cost planning. The exact answer to Question 2 depends on the obstacle grid, which is not visible in the uploaded assessment.